Cursor Build Kit — TourWise AI Faith Travel Conversion

Paste this entire file into Cursor as your task prompt. Cursor will work through it sequentially.

CONTEXT (read first)

You are working on the existing TourWise AI Next.js site at `tourwiseai.com`. This site is currently a generic AI travel planner with low traffic. We are pivoting it to lead with **Faith-Based Travel** as our primary affiliate revenue niche while keeping the generic AI planner intact for other use cases.

We are an affiliate site monetized through Travelpayouts (which covers Booking.com, GetYourGuide, Tiqets, Viator, Discover Cars, Kiwi.com, and ~100 other travel brands under one platform), plus Amazon Associates for travel gear.

The end goal of this build: every page has clear affiliate-link placement, every outbound link is tracked, every visitor either books through us, joins our email list, or both. Nothing else matters.

Work through the steps in order. Do not skip ahead. After each major step, run the project and verify before moving to the next.

STEP 1 — Inventory and Setup

1. Open the project. Identify whether it uses Next.js App Router or Pages Router. (Assume App Router unless proven otherwise.)

2. Identify the existing styling approach (Tailwind, CSS Modules, etc.) and match it. Do not introduce new styling systems.

3. Confirm Node version, package manager (npm/pnpm/yarn), and the deployment target (Vercel).

4. Add the following dependencies if not already present:

@vercel/analytics

resend

@supabase/supabase-js

zod

5. Create a `.env.local` template in the project root (do not commit real values):

   NEXT_PUBLIC_GA_ID=

   NEXT_PUBLIC_SITE_URL=https://www.tourwiseai.com

   TRAVELPAYOUTS_MARKER=

   TRAVELPAYOUTS_API_TOKEN=

   RESEND_API_KEY=

   RESEND_FROM_EMAIL=hello@tourwiseai.com

   SUPABASE_URL=

   SUPABASE_SERVICE_ROLE_KEY=

STEP 2 — Folder Structure (create these)

src/

app/

faith-travel/page.tsx

holy-land-tours-from-usa/page.tsx

camino-de-santiago-planner/page.tsx

vatican-rome-christian-itinerary/page.tsx

affiliate-disclosure/page.tsx

```
    blog/[slug]/page.tsx
    api/
      lead-magnet/route.ts
      affiliate-click/route.ts
  components/
    AffiliateLink.tsx
    StickyTripCTA.tsx
    LeadMagnetForm.tsx
    AffiliateDisclosure.tsx
    ComparisonTable.tsx
    ItineraryCard.tsx
    FAQAccordion.tsx
  lib/
    affiliate.ts
    analytics.ts
    schema.ts
content/
  posts/
public/
  lead-magnets/
```

STEP 3 — `src/lib/affiliate.ts` (the centerpiece)

Create this file. Every affiliate link in the entire site flows through it. No exceptions.

```
// src/lib/affiliate.ts

const MARKER = process.env.NEXT_PUBLIC_TRAVELPAYOUTS_MARKER ||
process.env.TRAVELPAYOUTS_MARKER || "";

export type AffiliateProgram =

  | "booking"

  | "getYourGuide"

  | "tiqets"

  | "viator"

  | "discoverCars"

  | "kiwi"

  | "tripCom"

  | "wayAway"

  | "hotellook"

  | "klook"

  | "amazon";

interface ProgramConfig {

  baseUrl: string;

  buildUrl: (path: string, subid: string) => string;

}

const PROGRAMS: Record<AffiliateProgram, ProgramConfig> = {
```

```
  booking: {
    baseUrl: "https://www.booking.com",
    buildUrl: (path, subid) => {
      const url = new URL(path, "https://www.booking.com");
      url.searchParams.set("aid", MARKER);
      url.searchParams.set("label", subid);
      return url.toString();
    },
  },
  getYourGuide: {
    baseUrl: "https://www.getyourguide.com",
    buildUrl: (path, subid) =>
`https://www.getyourguide.com${path}?partner_id=${MARKER}&utm_medium=online_publisher
&placement=tourwise&cmp=${subid}`,
  },
  tiqets: {
    baseUrl: "https://www.tiqets.com",
    buildUrl: (path, subid) =>
`https://www.tiqets.com${path}?partner=${MARKER}&utm_source=tourwise&utm_campaign=${
subid}`,
  },
```

```
  viator: {
    baseUrl: "https://www.viator.com",
    buildUrl: (path, subid) =>
`https://www.viator.com${path}?pid=${MARKER}&mcid=42383&medium=link&campaign=${subid}`,
  },
  discoverCars: {
    baseUrl: "https://www.discovercars.com",
    buildUrl: (path, subid) =>
      `https://www.discovercars.com${path}?a_aid=${MARKER}&a_bid=${subid}`,
  },
  kiwi: {
    baseUrl: "https://www.kiwi.com",
    buildUrl: (path, subid) =>
      `https://www.kiwi.com${path}?affilid=${MARKER}&subid=${subid}`,
  },
  tripCom: {
    baseUrl: "https://www.trip.com",
    buildUrl: (path, subid) =>
      `https://www.trip.com${path}?Allianceid=${MARKER}&SID=${subid}`,
  },
  wayAway: {
```

```
  baseUrl: "https://wayaway.io",
  buildUrl: (path, subid) =>
    `https://wayaway.io${path}?marker=${MARKER}.${subid}`,
},
hotellook: {
  baseUrl: "https://hotellook.com",
  buildUrl: (path, subid) =>
    `https://hotellook.com${path}?marker=${MARKER}.${subid}`,
},
klook: {
  baseUrl: "https://www.klook.com",
  buildUrl: (path, subid) =>
    `https://www.klook.com${path}?aid=${MARKER}&utm_campaign=${subid}`,
},
amazon: {
  baseUrl: "https://www.amazon.com",
  buildUrl: (path, subid) => {
    const tag = process.env.NEXT_PUBLIC_AMAZON_TAG || "tourwiseai-20";
    const url = new URL(path, "https://www.amazon.com");
    url.searchParams.set("tag", tag);
    url.searchParams.set("ascsubtag", subid);
```

```
      return url.toString();
    },
  },
};
export function buildAffiliateUrl(
  program: AffiliateProgram,
  path: string,
  subid: string
): string {
  return PROGRAMS[program].buildUrl(path, subid);
}
// SubID convention: <niche>-<page>-<position>
// Example: faith-holyland-comparison-row1, camino-planner-cta-bottom
export function makeSubId(niche: string, page: string, position: string): string {
  return `${niche}-${page}-${position}`.toLowerCase().replace(/\s+/g, "");
}
```

IMPORTANT: Note the env var is read as both `NEXT_PUBLIC_TRAVELPAYOUTS_MARKER` (needed for client components) and `TRAVELPAYOUTS_MARKER` (server). Set both to the same value in Vercel.

STEP 4 — `src/components/AffiliateLink.tsx`

```
"use client";
import { useCallback } from "react";
```

```
import { buildAffiliateUrl, type AffiliateProgram } from "@/lib/affiliate";

interface Props {
  program: AffiliateProgram;
  path: string;
  subid: string;
  children: React.ReactNode;
  className?: string;
}

export default function AffiliateLink({ program, path, subid, children, className }: Props) {
  const url = buildAffiliateUrl(program, path, subid);
  const handleClick = useCallback(() => {
    if (typeof window === "undefined") return;
    const payload = JSON.stringify({ program, subid, ts: Date.now() });
    if (navigator.sendBeacon) {
      navigator.sendBeacon("/api/affiliate-click", payload);
    } else {
      fetch("/api/affiliate-click", { method: "POST", body: payload, keepalive: true });
    }
    if ((window as any).gtag) {
      (window as any).gtag("event", "affiliate_click", { program, subid });
    }
```

```
  }, [program, subid]);

  return (
    <a
      href={url}
      target="_blank"
      rel="sponsored noopener noreferrer"
      className={className}
      onClick={handleClick}
    >
      {children}
    </a>
  );
}
```

Use this component for every outbound monetized link in the entire site. Refactor any existing hard-coded affiliate links to use it.

STEP 5 — Affiliate Disclosure (legal — non-negotiable)

`src/components/AffiliateDisclosure.tsx`

```
import Link from "next/link";

export default function AffiliateDisclosure() {
  return (
    <div className="text-xs text-gray-500 border-t pt-4 mt-8 max-w-3xl mx-auto px-4">
```

```
      <p>
        <strong>Affiliate Disclosure:</strong> TourWise AI participates in affiliate programs
        including Travelpayouts (covering Booking.com, GetYourGuide, Tiqets, Viator, and others)
        and Amazon Associates. When you click certain links and make a purchase, we may earn
a
        commission at no extra cost to you. This helps keep our AI travel planner free.{" "}
        <Link href="/affiliate-disclosure" className="underline">Read our full disclosure</Link>.
      </p>
    </div>
  );
}
```

Add `<AffiliateDisclosure />` to the root layout footer so it appears on every page.

src/app/affiliate-disclosure/page.tsx

Build a full disclosure page that covers FTC requirements, lists all programs, explains the relationship, and includes Texas DSPA-compliant language. Include sections: "How We Make Money," "Programs We Use," "Your Rights as a Reader," "Editorial Independence," "Contact Us." Minimum 600 words.

STEP 6 — Sticky Trip CTA

src/components/StickyTripCTA.tsx

```
"use client";

import Link from "next/link";

import { useState, useEffect } from "react";
```

```
interface Props {
  destination?: string;
}
export default function StickyTripCTA({ destination }: Props) {
  const [visible, setVisible] = useState(false);
  useEffect(() => {
    const onScroll = () => setVisible(window.scrollY > 400);
    window.addEventListener("scroll", onScroll);
    return () => window.removeEventListener("scroll", onScroll);
  }, []);
  if (!visible) return null;
  const href = destination
    ? `/?destination=${encodeURIComponent(destination)}`
    : "/";
  return (
    <div className="fixed bottom-4 inset-x-4 md:inset-x-auto md:right-6 md:max-w-sm z-50">
      <Link
        href={href}
        className="block bg-amber-600 hover:bg-amber-700 text-white font-semibold py-4 px-6
rounded-xl shadow-2xl text-center transition"
        onClick={() => {
          if ((window as any).gtag) {
```

```
          (window as any).gtag("event", "cta_click", { source: "sticky", destination });
        }
      }}
    >
      Plan My Free {destination ? `${destination} ` : ""}Itinerary →
    </Link>
  </div>
 );
}
```

Add to the root layout. Pass `destination` prop on niche pages so the CTA pre-fills the planner.

STEP 7 — Lead Magnet Form + API

`src/components/LeadMagnetForm.tsx`

Build a form with: name (optional), email (required), and a hidden field for `magnet_id`. On submit, POST to `/api/lead-magnet`. Show success state with download link.

`src/app/api/lead-magnet/route.ts`

```
import { NextRequest, NextResponse } from "next/server";
import { Resend } from "resend";
import { z } from "zod";
import { createClient } from "@supabase/supabase-js";

const schema = z.object({
  email: z.string().email(),
```

```
  name: z.string().optional(),
  magnet_id: z.string(),
});
export async function POST(req: NextRequest) {
  const body = await req.json();
  const parsed = schema.safeParse(body);
  if (!parsed.success) return NextResponse.json({ error: "Invalid input" }, { status: 400 });
  const { email, name, magnet_id } = parsed.data;
  // Store in Supabase
  const supabase = createClient(
    process.env.SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );
  await supabase.from("leads").insert({ email, name, magnet_id, source: "tourwise" });
  // Send the magnet via email
  const resend = new Resend(process.env.RESEND_API_KEY);
  const downloadUrl =
`${process.env.NEXT_PUBLIC_SITE_URL}/lead-magnets/${magnet_id}.pdf`;
  await resend.emails.send({
    from: process.env.RESEND_FROM_EMAIL!,
    to: email,
    subject: "Your free TourWise AI itinerary is ready",
```

```
    html: `<p>Hi${name ? ` ${name}` : ""},</p><p>Your free itinerary is ready: <a href="${downloadUrl}">Download it here</a>.</p><p>Reply to this email if you have any questions about planning your trip.</p>`,

  });

  return NextResponse.json({ ok: true, downloadUrl });

}
```

Run this Supabase migration:

```
create table if not exists leads (

  id uuid primary key default gen_random_uuid(),

  email text not null,

  name text,

  magnet_id text not null,

  source text default 'tourwise',

  created_at timestamptz default now()

);

create index leads_email_idx on leads (email);
```

STEP 8 — Affiliate Click Logging

src/app/api/affiliate-click/route.ts

```
import { NextRequest, NextResponse } from "next/server";

import { createClient } from "@supabase/supabase-js";

export async function POST(req: NextRequest) {
```

```
  try {
    const text = await req.text();
    const data = text ? JSON.parse(text) : {};
    const supabase = createClient(
      process.env.SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );
    await supabase.from("affiliate_clicks").insert({
      program: data.program,
      subid: data.subid,
      user_agent: req.headers.get("user-agent") || "",
      referer: req.headers.get("referer") || "",
    });
  } catch (e) {
    // fail silently — never block the user's redirect
  }
  return NextResponse.json({ ok: true });
}
create table if not exists affiliate_clicks (
  id uuid primary key default gen_random_uuid(),
  program text not null,
```

```
  subid text not null,
  user_agent text,
  referer text,
  created_at timestamptz default now()
);
create index affiliate_clicks_subid_idx on affiliate_clicks (subid);
create index affiliate_clicks_created_at_idx on affiliate_clicks (created_at);
```

STEP 9 — Money Page 1: `/holy-land-tours-from-usa`

Build `src/app/holy-land-tours-from-usa/page.tsx` with this exact structure:

1. **H1:** "Holy Land Tours from USA — 2026 Complete Guide & Comparison"
2. **Meta description:** "Compare the best Christian Holy Land tours departing from US cities in 2026. Prices, itineraries, and what to know before you book your pilgrimage to Israel."
3. **Hero block:** brief intro paragraph + first CTA (`<StickyTripCTA destination="Holy Land" />` is already global, but add an inline button here too: "Build My Custom Holy Land Itinerary").
4. **Quick summary box** (3 bullets — for scanners):
   - Holy Land tours from USA range $3,048–$4,998 starting price (2026 data)
   - Most tours are 8–12 days with group sizes 16–30
   - Departures available year-round, but Easter / Christmas book up 6+ months ahead
5. **Comparison table** (`<ComparisonTable />`) with at least 5 reputable tour operators. Use real, current data: ETS Tours, Pilgrim Tours, Inspiration Travel, Immanuel Tours, Elbow Holy Land Tours. Columns: Tour Name, Days, Price From, Group Size, Departure Cities, Highlights, Link. Each link uses `<AffiliateLink>` where the operator participates in Travelpayouts; for non-affiliate operators, link directly with rel="nofollow" (we still mention them for editorial credibility).
6. **Long-form content** (1500-2000 words) covering:
   - "What Makes a Good Holy Land Tour" (criteria for choosing)
   - "Best Time to Go" (with seasonality and price implications)
   - "Hotels in Jerusalem and Tel Aviv" — embed 3-4 `<AffiliateLink program="booking" />` links to specific Jerusalem hotels

    - "Day Tours and Skip-the-Line Tickets" — embed 3-4 `<AffiliateLink program="getYourGuide" />` and `<AffiliateLink program="tiqets" />` links
    - "Travel Insurance for Israel" — affiliate link to InsureMyTrip or Insubuy
    - Naturally embedded affiliate links — never list-style "click here links," always in-context recommendations
7. **Lead magnet block:** "Get the free 12-day Holy Land itinerary PDF" → `<LeadMagnetForm magnet_id="holy-land-12-day" />`
8. **FAQ section** (`<FAQAccordion />`) with at least 10 questions:
    - Is it safe to visit Israel in 2026?
    - How much does a Holy Land tour cost?
    - What should I pack?
    - Can seniors do the tour?
    - What's the difference between Catholic and Protestant Holy Land tours?
    - Do I need a visa?
    - Can I add Petra to my trip?
    - How do I choose between operators?
    - Is travel insurance required?
    - When should I book?
9. **Schema markup:** `Article` + `FAQPage` (use `lib/schema.ts` helpers).
10. **Footer CTA:** repeat the planner CTA + lead magnet block.

SubID convention for this page: all links use prefix `faith-holyland-` followed by position (e.g., `faith-holyland-comparison-row1`, `faith-holyland-hotel-jerusalem-king-david`, `faith-holyland-tiqets-vatican-walls`).

STEP 10 — Money Page 2: `/camino-de-santiago-planner`

Same template structure. Lead with comparison of the major Camino routes (French Way, Portuguese Way, Northern Way, Primitive Way), embedded `<AffiliateLink program="booking" />` to albergues in major stops (Pamplona, Burgos, León, Sarria, Santiago de Compostela), `<AffiliateLink program="discoverCars" />` for travelers driving sections, and `<AffiliateLink program="amazon" />` for the gear list (boots, poncho, backpack, blister kit, headlamp).

Lead magnet: "Free 7-day Camino Frances itinerary PDF for older walkers" **Bottom CTA:** "Plan my custom Camino itinerary →" pre-fills planner with destination=Camino de Santiago.

SubID convention: `faith-camino-` prefix.

STEP 11 — Money Page 3: `/vatican-rome-christian-itinerary`

Same template. Lead with a 5-day Christian-focused Rome itinerary (Vatican Day, Christian Rome day, Catacombs day, Day Trip to Assisi, Day Trip to Florence/Pompeii). Heavy on `<AffiliateLink program="tiqets" />` and `<AffiliateLink program="getYourGuide" />` for skip-the-line Vatican / St. Peter's / Sistine Chapel / Catacombs tickets. Hotel recommendations near the Vatican via `<AffiliateLink program="booking" />`.

Lead magnet: "Free 5-day Christian Rome itinerary PDF" **SubID convention:** `faith-vatican-` prefix.

STEP 12 — Hub Page: `/faith-travel`

A simple landing page that explains our faith-travel angle and links to the three money pages with hero cards. Include a bonus "Coming Soon" section listing future destinations (Camino Portuguese, Israel for Pastors, Reformation Tour Germany, Greece + Turkey Footsteps of Paul) — visitors can sign up for "Notify Me" which captures their email for the future content.

STEP 13 — Schema Helpers

`src/lib/schema.ts`

```
export function articleSchema(opts: {

  title: string;

  description: string;

  url: string;

  datePublished: string;

  dateModified: string;
```

```
  imageUrl: string;
}) {
  return {
    "@context": "https://schema.org",
    "@type": "Article",
    headline: opts.title,
    description: opts.description,
    url: opts.url,
    datePublished: opts.datePublished,
    dateModified: opts.dateModified,
    image: opts.imageUrl,
    author: { "@type": "Organization", name: "TourWise AI" },
    publisher: {
      "@type": "Organization",
      name: "TourWise AI",
      logo: { "@type": "ImageObject", url: `${process.env.NEXT_PUBLIC_SITE_URL}/logo.png` },
    },
  };
}
export function faqSchema(items: { q: string; a: string }[]) {
  return {
    "@context": "https://schema.org",
```

```
    "@type": "FAQPage",
    mainEntity: items.map(({ q, a }) => ({
      "@type": "Question",
      name: q,
      acceptedAnswer: { "@type": "Answer", text: a },
    })),
  };
}
export function touristTripSchema(opts: {
  name: string;
  description: string;
  url: string;
  itinerary: string[];
}) {
  return {
    "@context": "https://schema.org",
    "@type": "TouristTrip",
    name: opts.name,
    description: opts.description,
    url: opts.url,
    itinerary: opts.itinerary.map((step, i) => ({
```

```
      "@type": "TouristAttraction",
      position: i + 1,
      name: step,
    })),
  };
}
```

Embed via `<script type="application/ld+json" dangerouslySetInnerHTML={{ __html: JSON.stringify(schema) }} />` in each page.

STEP 14 — Analytics + GA4 Integration

In `src/app/layout.tsx`, inject GA4:

```
import Script from "next/script";

// inside <head>:

<Script src={`https://www.googletagmanager.com/gtag/js?id=${process.env.NEXT_PUBLIC_GA_ID}`} strategy="afterInteractive" />

<Script id="ga-init" strategy="afterInteractive">{`
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', '${process.env.NEXT_PUBLIC_GA_ID}');
`}</Script>
```

Add gating: only fire in production (`process.env.NODE_ENV === "production"`).

STEP 15 — Sitemap + robots.txt Updates

Update `src/app/sitemap.ts` to include all new routes. Verify `robots.txt` allows all crawlers but blocks `/api/` and any preview subdomain.

STEP 16 — Homepage Tweaks

Do not rebuild the homepage. Make these targeted changes:

1. Add a new section above the fold or just below the main hero: "Planning a faith-based trip?" → CTA to `/faith-travel`.
2. Replace generic "AI travel planner" hero copy with: "AI-powered trip planning for travelers who want more than the tourist trail."
3. Add a row of three featured destination cards linking to the three money pages.

STEP 17 — Lead Magnet PDFs

Create three PDF templates. For now, use placeholder content (we'll refine the actual itineraries later). Save to `public/lead-magnets/`:

- `holy-land-12-day.pdf`
- `camino-7-day.pdf`
- `vatican-5-day.pdf`

If there's a `pdfkit` or `react-pdf` lib already in the project use that; otherwise create simple branded PDFs from the existing money page content.

STEP 18 — Post-Build Verification Checklist

Before committing as done, manually verify:

- ☐ Click an affiliate link on each money page. Confirm it opens in a new tab with the correct affiliate marker in the URL.
- ☐ Confirm the click was logged in Supabase `affiliate_clicks` table.

- [ ] Submit the lead magnet form. Confirm:
  - Lead row written to Supabase
  - Email arrives in inbox with download link
  - Download link works
- [ ] On mobile (Chrome DevTools mobile mode), the sticky CTA appears after scrolling 400px.
- [ ] Run Lighthouse on each money page. Target ≥ 80 mobile performance, ≥ 90 SEO, ≥ 90 accessibility.
- [ ] Validate schema with Google's Rich Results Test (https://search.google.com/test/rich-results) on each money page URL.
- [ ] Verify `<AffiliateDisclosure />` appears in the footer of every page.
- [ ] Check `/affiliate-disclosure` page exists and contains all required sections.
- [ ] All affiliate links open with `rel="sponsored noopener noreferrer"` and `target="_blank"`.
- [ ] No hard-coded affiliate URLs anywhere in the codebase — grep for `booking.com`, `getyourguide.com`, etc., and confirm every match goes through `buildAffiliateUrl`.
- [ ] Submit updated sitemap to Google Search Console.

STEP 19 — Deploy

1. Push to main branch.
2. Verify Vercel deployment succeeds.
3. Set all production env vars in Vercel dashboard.
4. Submit the four new URLs to Google Search Console for indexing:
   - `/faith-travel`
   - `/holy-land-tours-from-usa`
   - `/camino-de-santiago-planner`
   - `/vatican-rome-christian-itinerary`

NOTES FOR CURSOR

- Match the existing project's code style. Don't introduce ESLint/Prettier configs that conflict with what's there.
- If a component already exists with a similar purpose, extend it rather than creating a duplicate.
- Use TypeScript strictly — no `any` types except where unavoidable (third-party libs).
- All copy in the money pages must be original. Do NOT scrape competitor content.

- If you're unsure about a design decision, default to "what would convert better?" not "what looks cleaner?"
- When done, summarize what you built, what was skipped, and any open questions in a `BUILD_NOTES.md` file at the project root.